



The Effects of Using Different Tools in Programming Teaching of Secondary School Students on Engagement, Computational Thinking and Reflective Thinking Skills for Problem Solving

Hatice Yildiz Durak¹

Published online: 21 August 2018
© Springer Nature B.V. 2018

Abstract

The aim of this research is to determine the effects of Scratch and Alice tools and programming teaching practices on student engagement, reflective thinking and problem-solving skills and computational thinking (CT) comparatively. A quasi-experimental design was used in the research and patterns with pretest–posttest control group were chosen for the variables. The study group was compromised of 110 students at 5th grade in 2016 spring semester. 1st and 2nd groups of Computing Technologies Course have been assigned objectively. Alice programming tool was preferred in the learning process of the 1st experimental group, whereas, Scratch programming tool was utilised in the 2nd study group during application process which lasted for 8 weeks. Results of the research show that programming teaching with Scratch has affected engagement and reflective thinking skills of the students for problem solving more positively than Alice. It has been stated that teaching with Alice affects skills related to CT of the students positively. Furthermore, some recommendations have been made for the future researches.

Keywords K-12 · Programming teaching · Alice · Scratch · Reflective thinking for problem solving · Computational thinking · Student engagement

1 Introduction

When programming teaching is considered, generally higher education students come to mind as the target audience (Yildiz Durak 2018). However, programming teaching has been integrated into curriculums in secondary schools and even in elementary school and nursery classes along with various applications for gaining twenty-first century qualifications in education (Fessakis et al. 2013).

✉ Hatice Yildiz Durak
hatyil05@gmail.com

¹ Department of Computer Education and Instructional Technology, Faculty of Education, Bartın University, Bartın, Turkey

Denner et al. (2012) have argued that activities conducted during teaching programming and products are oriented towards developing and reflecting problem solving as well as CT skills. For this reason, it is important to utilise the tools that help students develop their creativity and communication skills in programming. Within this context, the utilisation of different programming tools in teaching programming will enable students to develop their skills, including problem solving and CT, to enable students to reflect their ideas creatively through different tools and to apply the skills of using information technology to the application of programming concepts with different tools (Fields et al. 2014).

The active and sustainable engagement of students, on the other hand, is significant in teaching programming (Yildiz Durak 2018). Otherwise, it is inevitable that students are going to have difficulty in programming, which is a complex process requiring lots of skills. As a result of the difficulties experienced by students, the learning difficulties of the students increase; this affects the achievements, the interest and the affective reactions of the students adversely (Denner et al. 2012; Robins et al. 2003). The utilisation of different tools during teaching programming may have different effects on students' interest regarding to the course.

Students have various difficulties in programming teaching, which is a complex process that requires many skills. Results of studies done in computer science show that students are lost in codes in programming processes and drowned in details. This is why students focus on codes rather than design (Bucci et al. 2001). Berge et al. (2003) draw attention to the importance of programming tool in programming language. Considering these problems, it is obvious that using a tool to help learners to understand programming logic and thinking skills of the twenty-first century in an easier way and to contribute to the solution of problems which will be experienced in the programming education in early age is necessary. It is seen that different tools have been developed and different methods and approaches have been tested for the solution of the mentioned problems in literature (Trees 2010). When existing studies in this area are examined, it is concluded that especially visualization tools have been used to solve problems of programming teaching and to develop various skills. As a result of conducted literature scanning, it is stated that these tools support the qualifications such as algorithmic thinking, reflective thinking and problem solving skills besides their positive effects to engagement (Wang et al. 2009). Alice and Scratch tools are frequently used in studies in this context (Brennan 2013; Kayabaşı 2016). The main problem here is which tool is more efficient for twenty-first century skills of the students. For this reason, it is essential to search for which programming tool has more positive effects for engagement of students and developing reflective thinking skill for problem solving, CT skill.

1.1 Purpose of the Research

The aim of this research is to determine the effects of Scratch and Alice tools and programming teaching practices on student engagement, reflective thinking for problem-solving skills and CT comparatively. To achieve these objectives, answers to the following questions have been sought.

- What level are the students' engagement, reflective thinking for problem solving and CT skills before and after teaching programming by means of Scratch and Alice?

- Is there any significant difference between the practice of programming with Scratch and Alice and the level of students' participation to the course, reflective thinking for problem solving and CT when pre-test scores are examined?

2 Conceptual Framework

2.1 The Utilisation of Different Programming Tools in Teaching Programming

Teaching practices regarding programming focusing on developing children's skills have boosted over the last 10 years (Al-Jarrah 2016). In line with this trend, attempts directed to designing different programming tools capture the attention of public (Durak 2016). Accordingly, it has been stressed in the existing literature that expected learning outcomes haven't been achieved yet and there are some difficulties related to teaching programming (Esteves and Mendes 2004; Hongwarittorn and Krairit 2010).

Students generally consider programming as a complex, difficult task which is hard to achieve (Denner et al. 2012; Fesakis and Serafeim 2009; Robins et al. 2003). This perception diminishes the motivation and participation to the course. Forte and Guzdial (2014) argues that the main reason for failure in teaching programming is the lack of interest in programming lessons owing to early abandonment and programming lessons because of negative considerations about programming. These problems in teaching programming lead to the doubts about the effectiveness of the programming tools used (Forte and Guzdial 2014).

Margulieux et al. (2012) emphasizes that the difficulties in programming can be overcome by turning it into an easier and more enjoyable experience with the help of different programming environments. Resnick et al. (2009) mentions that the development of programming tools reducing the cognitive burden that motivates new learners to learn programming, and the provision of continuity of this development may be a solution. For this reason, various environments and tools have been started to be utilised to enable children to comprehend programming and to involve in programming practices.

When dealt with a broad perspective, programming environment and tools can be classified as text-based and block-based (Yıldız Durak and Güyer 2018). It is necessary to use text-based characters in accordance with syntax rules of programming while coding in text-based programming, whereas block-based programming environments are typically those including visual programming. In these kinds of environments, the coding process can be performed with drag-and-drop method of the visual objects on the screen, without having to write a code according to the syntax rules of the programming language.

In recent years, it has been witnessed that tools such as Scratch, Alice and App Inventor have made great contributions to teaching programming to children (no mistake while coding or easy detection, no syntax errors, easy to debug, designing projects with multimedia tools, reducing cognitive load, simple and easy-to-use interface, etc.) and have been used prevalently (Durak 2016). These programming tools have the structure that makes easy to understand and use the basic algorithmic forms for new beginners in programming as well as prevent the complexity of syntax in programming, featuring the visual sides of programming (Chang 2014; Portelance and Bers 2015).

2.2 Why are Scratch and Alice?

Scratch and Alice are two of the most known and used programming languages in block-based programming environments (Medlock-Walton et al. 2014). One of the reasons for choosing these two programming tools in this present study is conclusive evidence that these tools are more beneficial in forming positive perceptions about programming than others. In addition, in a practice conducted in a secondary school affiliated to Ministry of National Education (MoNE) in Turkey, some content and materials were provided for Scratch (<http://scratch.eba.gov.tr>) and Alice (<http://f.eba.gov.tr/Kaynak/alice>) in a national database called Education Information Network (EIN) by MoNE. Such a content or material in Turkish haven't been provided by MoNE for any other programming tools.

When the structure of Scratch and Alice programming tools are examined, both allow their users to design projects according to their preferences. The works of Papadakis et al. (2014) tell us that Alice is a more convenient decision than Scratch in that it emphasizes Java and Java related concepts. It was stated that Scratch caused less technical problems during the practice than Alice, however. In Scratch, it is possible to make arrangements in two-dimensional images, while a user can form three-dimensional images and can make arrangements when needed in Alice. Scratch concentrates on self-directed learning and enable its users to share projects with active user groups in online learning environments (Maloney et al. 2010). Thus, Scratch intends to increase the motivation of users by providing opportunities to learn from others. While both tools allow the transfer of various multimedia objects into the programming environment, Scratch includes tools for drawing images and recording sounds, which is important to support student engagement (Durak 2016).

2.3 The Effect of Using Different Programming Tools on Student Engagement in Teaching Programming

Engagement is a preliminary condition for learning (Fredricks et al. 2004). Cole and Chan (1994) defined the student engagement as “the student engagement and the degree of active engagement to learning activities”. Student engagement in classrooms where active learning processes are performed is seen as an important component of learning because of its educational benefits (Berman 2014; Kuh 2009; Lippmann 2013).

An active and sustainable engagement of the students to activities and learning process is significant in learning programming. Engagement is a variable that must not be neglected especially in courses such as programming in which students develop negative attitudes in many situations, especially when they consider it as a difficult task. Moreover, as indicated by Law et al. (2010), there is a linear relationship between active engagement and learning in programming education. It is stated that the lack of attendance of the course by the learners will make it almost impossible to acquire the programming skill. In short, it can be said that programming learning will take place more efficiently with the active engagement of the individual. Active engagement may be affected by the features of programming tools (Chang 2014), for students may have difficulty maintaining and participating in programming concepts due to problems (not being able to debug, complexity of tools, less intelligibility of interface, characters in the environment, commands, technical problems emerging while using programming tools, ...etc.) caused by the structural features of programming tools and environments (Winslow 1996).

2.4 The Effect of Using Different Programming Tools on Reflective Thinking Skills for Problem Solving

Reflection can be defined as the way in which the learners experience various ways of designing their own learning process as a result of their experiences (Prensky and Berry 2001). Reflective learning, on the other hand, is a powerful learning experience that supports understanding of content and knowledge transfer by examining content units with a critical perspective (Strampel and Oliver 2007).

Learners are provided with new opportunities to reflect their own learnings in learning and teaching processes thanks to different programming tools designed in teaching programming. It is not enough to know and implement only the programming codes while preparing the problem solutions in the programming process, however. Problem solving skills must be developed for learning programming steps (Antonakos 2016), for learning programming is considered as a process encompassing problem solving steps (Gomes and Mendes 2007).

Reflective thinking, however, emerges only when a specific problem is perceived (Shermis 1992). For this reason, it is obvious that reflection can be observed in the problem solving process (Kizilkaya and Askar 2009). The features offered by the different programming tools are issues to be explored whether they affect the ability of students to reflect on problem-solving processes. In this study, reflective thinking for problem solving is considered as an important skill in the programming learning process.

2.5 The Effect of Using Different Programming Tools on Computational Thinking Skills

CT encompasses understanding the problem, finding suitable solutions, thinking patterns such as abstracting and reflecting (Wing 2006). CT refers to having necessary knowledge, skill and attitudes in order to use computers with the purpose of solving problems encountered (Özden 2015). CT is a means of understanding human behaviours within the context of the basic terms of computing, system design and problem solving (Korkmaz et al. 2017). In addition, it also includes some skills such as data analysis, summarizing data and modeling. It is simply defined as a problem solving and formulating process by using computers.

Computer programming skill, considered as an important skill in present day, is a mutual strategy for developing CT skills (Fields et al. 2012; García-Peñalvo 2016; Grover and Pea 2013; Katai and Toth 2010; Lawanto et al. 2017; Lye and Koh 2014; Moreno 2012; Saritepeci and Durak 2017; Werner et al. 2012, 2015; Wing 2011). Programming education is seen as key to gaining skills for developing CT skills and also called twenty-first century skills such as creativity, critical thinking, problem solving, communication and collaboration, social-intercultural skills, productivity, leadership and responsibility (Lau and Yuen 2011; Yıldız Durak and Güyer 2018).

Since CT includes different high-level thinking skills and knowledge areas, the effectiveness of the programming processes in the development of these knowledge and skills enables the development of CT skills in children. On the other hand, it is a requirement for us to figure out the problems encountered by computers and various programs in our day and behave “like a computer” so as to find suitable solutions. Individuals need to know the computer language and have code literacy so that they can participate in daily life activities (Rushkoff 2010). It is believed that the use of different programming tools in the process

of early programming education may be influential on the level of development of various skills such as “behaving like a computer”. For this reason, the concept of CT has been included in the study.

3 Method

3.1 Research Pattern

This study was designed as a quasi-experimental one with pre-test and post-test control groups. Experimental pattern used in the research has been presented in Table 1.

3.2 The Study Group

The research was done with 110 participants who are students of the 5th grade in 2016 spring semester in Ankara. In line with the aim of the research, 5th grade pupils who didn't have any experience on programming and took the programming course for the first time were included in this study. The school defined as sampling of the study was selected via convenience sampling method. The socio-economic level of the region where the school was located and of the students were classified as “moderate level” by MoNE. Thus, it can be said that the sampling school isn't at the highest top level. The students taking the course of Information Technologies were classified as experiment and control groups impartially. In addition, pupils are distributed to the classes by the school management in such a way that they normally distribute in terms of academic achievement in all courses. Nevertheless, in this study, it was checked whether there were significant differences between the levels of the participants in terms of gender and academic achievement.

The 1st group consisted of 25 female and 30 male participants and 2nd group consisted of 31 female and 24 male participants. Considering the gender distribution of the students in the groups, it can be said that both groups have similar variable in terms of gender (Pearson Chi square: 1.310 $p \geq .05$). It can be said that groups have a similar distribution in terms of academic achievement (Pearson Chi square: 1.514 $p \geq .05$).

Table 1 Experimental design

Group	Assignment	Used tools/operations	Pretest	Posttest
G_1	U	Teaching programming with Alice (X_A)	$O_{1,1}$	$O_{2,1}$
			$O_{1,2}$	$O_{2,2}$
			$O_{1,3}$	$O_{2,3}$
G_2	U	Teaching programming with Scratch (X_S)	$O_{1,1}$	$O_{2,1}$
			$O_{1,2}$	$O_{2,2}$
			$O_{1,3}$	$O_{2,3}$

G_1 1st experimental group, G_2 2nd experimental group, U convenience sampling, X_A teaching programming with Alice, X_S teaching programming with Scratch, $O_{1,1}$ reflective thinking skill scale for problem solving—pretest, $O_{1,2}$ engagement scale—pretest, $O_{1,3}$ computational thinking scale—pretest, $O_{2,1}$ reflective thinking skill scale for problem solving—posttest, $O_{2,2}$ engagement scale—posttest, $O_{2,3}$ computational thinking scale—posttest

3.3 Operation

Practice of the research has been conducted in Computing Technologies and Software course of 5th grade in a secondary school affiliated to MoNE in Ankara during 8 weeks. In application process, Alice programming and related tools were used in the learning process of the 1st experimental group whereas Scratch programming and related tools were used for 2nd experimental group. The main reason why the 5th grade students were preferred is Computing Technologies and Software course given as a must course for the first time in this grade. Indeed, students in the study group were introduced programming, Alice and Scratch for the first time. In-class, on-line and face to face applications and tools applied/used in learning process used in practice are presented in Table 2.

3.4 Data Collection Tools

Three different data collection tools have been used in the research.

Personal Information Form: This form has been developed by researcher. Data related to personal information of participants (gender, participant number, information about experiences of computing technologies course and programming) has been collected via this data collection tool which is composed of 4 items. Form items differ according to questions and are generally likert-type.

Reflective Thinking Skills for Problem Solving Scale (RTSPS): This scale has been developed by Kızılkaya and Aşkar (2009). This scale, consisting of 14 items and 3 factors, is five-point Likert type scale. Three factors of this scale are “inquiry” consisting 5 items as sub-dimensions, “evaluation” consisting 5 items as sub-dimensions and “reasoning” consisting 4 items as sub-dimensions. In this research, cronbach alpha reliability coefficient calculated for the scale is high (.936). Cronbach alpha reliability coefficients for sub-dimensions are .858; .804; .790 in order.

Engagement Scale: This scale has been developed by Sartepeci (2012) within the scope of undergraduate thesis and validity and reliability have been studied. Items of this scale have been created by adapting items of National Survey of Student Engagement (NSSE) on-line scale for primary school level and examining existing studies about engagement of students. This scale, consisting of 34 items and 3 factors, is five-point Likert type scale. Five five-point Likert type scale varies from “Never(1)” to “Always(5)”. Three factors of this scale, “active learning and learning by assisting (ALLA)” sub-dimension consists 17 items, “student’s level of difficulty (SLD)” sub-dimension consists 7 items and “feedback level and teacher student interaction (FLTSI)” sub-dimension consists 10 items. In this research, cronbach alpha reliability coefficient calculated for the scale is high (.981). Cronbach alpha reliability coefficients for sub-dimensions are .967; .934; .953 in order.

Computational Thinking Scale: This scale has been developed by Korkmaz et al. (2015). This scale, which is originally aimed at university students, consists of 29 items and five factors. Then, adaption for secondary school has been again done by Korkmaz et al. (2015). This scale adapted to five-point Likert type scale and item number has decreased to 22. Five factors of this scale, “Creativity” sub-dimension consists 4 items, “Algorithmic Thinking” sub-dimension consists 4 items, “Collaboration” sub-dimension consists 4 items, “Critical Thinking” sub-dimension consists 4 items and “Problem Solving” sub-dimension consists 6 items. In this research, cronbach alpha reliability coefficient calculated for the scale is high (.866).

Table 2 Learning environment for students in experiment groups

Teaching programming with Alice		Teaching programming with Scratch	
Face-to-face components	Online components	Face-to-face components	Online components
Expression with interactive board Presentations about Alice Instructions (Installation direction, work-sheets, etc.) Examples for demonstration (http://www.alice.org) Questions and answers/discussions in the context of problems	Resources related to the course (presen-tation, internet page, multimedia etc.) Online individual events (blog, discus-sion, on-line research assignments)	Expression with interactive board Presentations about Scratch Instructions (Installation direction, work-sheets, etc.) Examples for demonstration (http://scratch.mit.edu) Questions and answers/discussions in the context of problems	Resources related to the course (presen-tation, internet page, multimedia etc.) multimedia items) Online individual events (blog, discussion, on-line research assignments)

3.5 Analysis of the Data

A Chi square test has been conducted to determine similarities or differences between gender-based distributions of students in the experimental groups. Before analysis, distribution of measures related to dependent variable was normal in both groups. In the analysis of data obtained after the experimental practice, ANCOVA test was conducted in order to determine the differences between groups. The assumptions of this test was checked before the analysis. Findings related to pre-test scores were examined and found no significant difference between pre-test related groups. ANCOVA was used to reduce the error variance and to remove the differences between the groups at the beginning of the practice (Mertler and Vannatta 2002), although there is no difference between the groups in terms of pre-test scores.

4 Findings

Findings are presented according to the order of sub-problems.

The arithmetic mean and standard deviation values of the data obtained from the engagement scale were calculated before and after teaching programming to determine engagement of students and are presented in Table 3.

When Table 3 is examined, it was found that the mean scores of the students' engagement to the course before the experimental practice was 63.07 in the group who was taught via Alice and it was 80.81 after the practice. On the other hand, the mean scores of the students' engagement to the course before the experimental practice was 61.78 in the group who was taught via Scratch and it was 89.23 after the practice. It was seen that the engagement scores in both group increased. However, it was found that pre-test scores increased by 17.74 points in the group with Alice and 27.45 points in Scratch group.

When the scores of students' reflective thinking skills for problem solving are examined, it is seen that the pre-test scores of the students who underwent programming learning process with Alice were 34.96 and the post test scores was found as 37.00. Similarly, in the group where Scratch was used, the pre-test scores of the students were 35.67, whereas the post test scores was found as 39.96. However, pre-test scores increased by 2.04 points in the group with Alice and 4.29 points in Scratch group.

The pre-test means of the students in the first experimental group were 70.18, and the post test scores were 79.42. In the second experimental group, CT pre-test scores of the students were 70.62 and post test scores were 77.66. However, there was an increase of 9.24 points in pre-test scores in the group with Alice and 7.04 points in Scratch group. Based on these findings, it can be said that programming with Alice and Scratch contributed to the positive increase in engagement, problem-solving reflective thinking and CT skill scores.

ANCOVA was used in order to test whether there is statistically significant difference between the engagement to the course scores and reflective thinking skills for problem solving and CT skills scores of the students. ANCOVA results are presented in Tables 4, 5 and 6 as to whether there is a significant difference between the scores of the post-test when pre-test scores are controlled.

As shown in Table 4, there is no statistically significant difference between post-test engagement scores adjusted for pre-test scores of the groups ($F(1-106)=1.155$,

Table 3 Levels of students lecture engagement, reflective thinking skills for problem solving and computational thinking

Scale	Group	Pretest		Posttest	
		Mean	SD	Mean	SD
Engagement	G1	63.07	35.49	80.81	42.87
	G2	61.78	35.36	89.23	40.88
ALLA	G1	21.05	13.38	40.16	22.35
	G2	27.76	20.73	45.05	21.63
SLD	G1	11.49	6.93	18.31	9.65
	G2	12.53	7.58	18.67	8.75
FLTISI	G1	17.18	9.31	22.34	12.91
	G2	17.78	8.88	25.51	12.26
RTSPS	G1	34.96	17.56	37.00	17.35
	G2	35.67	16.78	39.96	15.85
Inquiry	G1	11.60	5.97	13.09	7.05
	G2	11.65	4.39	14.18	6.01
Evaluation	G1	11.16	5.60	13.72	6.09
	G2	11.47	4.91	14.78	5.69
Reasoning	G1	8.71	4.90	10.18	5.11
	G2	10.35	4.36	11.00	4.92
CT	G1	70.18	11.06	79.42	14.66
	G2	70.62	11.98	77.66	13.60
Creativity	G1	9.60	5.50	16.02	8.07
	G2	9.84	5.55	15.92	7.92
Algorithmic thinking	G1	12.36	7.28	15.40	7.97
	G2	11.96	6.96	14.64	7.80
Collaboration	G1	12.28	8.40	16.40	9.11
	G2	12.08	8.29	15.60	8.97
Critical thinking	G1	8.64	7.15	15.84	8.38
	G2	8.36	7.05	14.80	8.13
Problem Solving	G1	14.46	8.19	15.60	9.01
	G2	14.76	8.86	16.44	9.22

ALLA active learning and learning by assisting, *SLD* student's level of difficulty, *FLTISI* feedback level and teacher student interaction, *RTSPS* reflective thinking skills for problem solving, *CT* computational thinking

$p > .05$). In this context, it can be said that the engagement levels of the students participating in the study with the different programming tools did not change significantly depending on the experimental process.

As shown in Table 5, there is a statistically significant difference between the post-test RTSPS scores adjusted for pre-test scores of the groups ($F(1-106) = 3.987, p \leq .05$). Within this context, it can be said that the RTSP skill levels of the students participating in the study changed significantly according to the experimental process which was conducted via different programming tools. The ANCOVA results are insufficient to determine the level of this significant effect. Eta-square (η^2) values were calculated to determine this effect size. Calculated Eta-square value was found as .043. Accordingly,

Table 4 ANCOVA results on differentiation of adjusted involvement post-test scores according to pre-test scores according to the programming tools used

Variables	Group	Mean	Adj. mean	Variance source	Sum of squares	df	F	Sig.	Partial Eta Sq.
<i>Engagement</i>	G1	80.81	80.82	Pretest	1220.949	1	1.155	.285	–
	G2	89.23	89.24	Method	2034.787	1			
				Error	186,785.918	106			
				Total	986,757.000				
<i>ALLA</i>	G1	40.16	40.16	Pretest	575.046	1	1.571	.213	–
	G2	45.05	45.06	Method	758.048	1			
				Error	51,145.122	106			
				Total	252,637.000				
<i>SLD</i>	G1	18.31	18.30	Pretest	60.955	1	1.186	.279	–
	G2	18.67	18.65	Method	101.042	1			
				Error	9034.435	106			
				Total	46,786.000				
<i>FLTSI</i>	G1	22.34	22.35	Pretest	9.951	1	.352	.554	–
	G2	25.51	25.53	Method	56.610	1			
				Error	17,054.415	106			
				Total	80,366.000				

ALLA active learning and learning by assisting, *SLD* student's level of difficulty, *FLTSI* feedback level and teacher student interaction

Table 5 ANCOVA results on differentiation of RTSPS post-test scores according to pre-test scores according to the programming tools used

Variables	Group	Mean	Adj. mean	Variance source	Sum of squares	df	F	Sig.	Partial Eta Sq.
<i>RTSPS</i>	G1	34.96	36.98	Pretest	158.757	1	3.987	.031	.043
	G2	35.67	39.91	Method	975.011	1			
				Error	28,809.678	106			
				Total	192,967.000				
<i>Inquiry</i>	G1	11.60	13.08	Pretest	27.621	1	3.690	.034	.041
	G2	11.65	14.161	Method	148.177	1			
				Error	4487.374	106			
				Total	25,132.000				
<i>Evaluation</i>	G1	11.16	13.72	Pretest	8.643	1	3.660	.035	.040
	G2	11.47	14.77	Method	123.555	1			
				Error	3635.973	106			
				Total	26,142.000				
<i>Reasoning</i>	G1	8.71	10.18	Pretest	19.398	1	3.650	.038	.038
	G2	10.35	10.99	Method	62.993	1			
				Error	2654.365	106			
				Total	15,083.000				

RTSPS reflective thinking skills for problem solving

it can be said that 4.3% of the variance observed in reflective thinking skill scores for problem solving stems from teaching programming via different programming tools. In short, according to the result of effect size, it can be said that programming via different programming tools has a “small effect” (Cohen, 1988) on students' reflective thinking skill scores for problem solving (for all subscales).

Table 6 ANCOVA results on differentiation of CT post-test scores according to pre-test scores according to the programming tools used

Variables	Group	Mean	Adj. mean	Variance source	Sum of squares	df	F	Sig.	Partial Eta Sq.
<i>CT</i>	G1	79.42	79.43	Pretest	1.564	1	.204	.653	–
	G2	77.66	77.67	Method	.098	1			
				Error	46.325	106			
				Total	1448.281				
Creativity	G1	16.02	16.03	Pretest	4.517	1	.211	.647	–
	G2	15.92	15.91	Method	.204	1			
				Error	102.425	106			
				Total	1881.813				
Algorithmic thinking	G1	15.40	15.40	Pretest	3.750	1	.165	.686	–
	G2	14.64	14.64	Method	.127	1			
				Error	81.691	106			
				Total	1645.125				
Collaboration	G1	16.40	16.39	Pretest	5.885	1	.043	.836	–
	G2	15.60	15.59	Method	.045	1			
				Error	110.837	106			
				Total	1880.438				
Critical thinking	G1	15.84	15.83	Pretest	3.608	1	.051	.822	–
	G2	14.80	14.81	Method	.041	1			
				Error	84.792	106			
				Total	1699.938				
Problem Solving	G1	15.60	15.61	Pretest	1.011	1	.062	.804	–
	G2	16.44	16.45	Method	.078	1			
				Error	133.992	106			
				Total	923.111				

CT computational thinking

As shown in Table 6, there is no statistically significant difference between the post-test CT scores adjusted for pre-test scores of the groups ($F(1-106) = .204, p > .05$). Within this context, it can be said that the CT skill levels of the students participating in the study with different programming tools did not change significantly depending on the experimental process.

5 Results and Discussion

The aim of this research is to compare the effects of Scratch and Alice tools and programming teaching practices on student engagement, reflective thinking toward problem-solving skills and CT.

According to results of the research, programming teaching with Scratch has affected reflective thinking skills toward problem solving of the students more positively than Alice. Furthermore, a significant difference has appeared between scores of reflective thinking skills about problem solving of the students depending on used programming tools. As indicated by Perkins and Salomon (1992), the process of activating the previous learning experience and transferring knowledge must be conducted so that problem solving and reflective thinking can be achieved successfully. From this point, it can be said that in the

process of developing reflective thinking skills for problem solving in the programming process of students, the use skills of previous information technologies will be effective, as well. Indeed, the skill level of information technology plays an important place in programming learning process (Durak 2016). The fact that Alice's program is more complex than Scratch can be considered as the reason for this finding in the current research.

Similarly, according to Maloney et al. (2010), main purpose of Scratch is to provide a single-user interface layout in single-window and simple command set which has language options and users can make design decisions (for example, selection of visual blocks) for beginner programmers who had no previous programming experience. Thus, Scratch programming media and its visual design allow children to express themselves easily and provide ways that facilitates their programming learning. In addition, Scratch's on-line web page is thought to provide students with the opportunity to share projects and feedback, supporting reflective thinking skills for problem solving. In a study conducted by Chang (2014), it was emphasized that the fact that the object design and stage adjustments of Alice are complex for new programmers may make concentrating programming problems difficult.

According to results of the research, programming teaching with Scratch has affected engagement of the students more positively than Alice. Scratch is more unsophisticated than Alice in terms of code blocks. Therefore, it can be said that students understand Scratch more easily affects active engagement positively. In support of this situation, it is emphasized that Scratch is quite suitable for children's coding teaching with simple code blocks, which makes it easy to use and allows less experienced users to concentrate on designing and producing without faltering coding mistakes (Durak 2016; Grover and Pea 2013; Sáez-López et al. 2016). According to Trees (2010), programming tools thought to be effective provide a manageable, flexible and visual media, support active engagement to learning activities and consist of simple items. These tools allow transfer of information to subsequent classes and provide technical and resource materials.

On the other hand, it was found that the more programming environment becomes complicated, the more possibly students may leave the programming education (Hill 2015; Noble 2013). For this reason, it is believed that 3D environments such as Alice have different points of interest of the students and may prevent active engagement in the classroom. Accordingly, Resnick et al. (2009) argued that Scratch reduce the cognitive load thanks to its simple interface.

In the study by Chang (2014), it was found that the group who learned via the Scratch program is more successful in the programming concepts, although the programming through Alice program increases the engagement levels by reducing the programming concerns of the students.

In the online environment provided by the Scratch program, on the other hand, the ability to easily share projects with the active user community provides advantages for students in increasing their learning motivation and in ensuring course engagement (Maloney et al., 2010). In addition, Papadakis et al. (2014) stated that the use of Scratch program is less technical knowledge and problem (installation, java installation, etc.) than Alice program. It can be said that the result of this study is based on technical knowledge and problems. It has been observed that programming with Alice affects skills related to CT besides problem solving of the students positively. Visual programming tools are frequently used in primary school level with purpose of teaching CT (Armoni et al. 2015; Bers et al. 2014). Especially programming activities starting at early ages has positive contribution to cognitive development of children (Akpınar and Altun 2014; Atmatzidou and Demetriadis 2016). Learning how to do programming requires strategy, planning, and logical thinking

skills, and therefore Programming tools must provide a fruitful environment for improving problem-solving skills, high-level thinking and metacognitive skills (Lavonen et al. 2003; Psycharis and Kallia 2017).

Thus, it is emphasized that 3D programming activities with Alice will help students to develop a lot of skills by providing opportunities for creativity, analytical and critical thinking skills, developing algorithmic thinking habits by producing solutions to complex problems (Akpınar and Altun 2014; Çakiroğlu et al. 2011; Fesakis and Serafeim 2009). According to Utting et al. (2010), it is important to provide an exciting and stimulating environment for students and to use the context of animation in applications based on algorithmic thinking, designing programs to solve creative problems and solving a problem. For this reason, they mention Alice is the most appropriate tool in especially STEM classes. In addition, it is said that Alice encourages students to develop their twenty-first-century skills harder.

5.1 Limitations

It was observed that some students had difficulties (especially in installing Alice program outside of school) because programming tools have different technical installations and usage requirements. The interest of the programs was different in the process. 3D structure of Alice programming was initially more interesting. However, it can be stated that effect of this innovation disappeared later. On the other hand, Alice was insufficient in terms of Turkish resources and samples.

5.2 Recommendations

It is suggested that research about CT for programming, learning environments for developing problem solving skills should be done. For example, programming teaching process provided by Scratch may be designed by various activities developing problem solving skill.

The differentiation appeared in engagement of students, CT, reflective thinking skills for problem solving as a result of Alice and Scratch tools usage can be examined deeply by doing a qualitative research to determine which characteristics of them cause this differentiation.

Effects of different programming tools except Alice and Scratch programming tools on CT, problem solving skills with different experimental applications can be examined.

References

- Akpınar, Y., & Altun, A. (2014). Bilgi toplumu okullarında programlama eğitimi gereksinimi. *İlköğretim Online*, 13(1), 1–4.
- Al-Jarrah, A. A. M. (2016). *Collaborative virtual environments for introductory programming*. Doctoral thesis, New Mexico State University, New Mexico.
- Antonakos, J. L. (Ed.). (2016). *Computer technology and computer programming: Research and strategies*. Boca Raton: CRC Press.
- Armoni, M., Meerbaum-Salant, O., & Ben-Ari, M. (2015). From scratch to “real” programming. *ACM Transactions on Computing Education (TOCE)*, 14(4), 25.
- Atmatzidou, S., & Demetriadis, S. (2016). Advancing students’ computational thinking skills through educational robotics: A study on age and gender relevant differences. *Robotics and Autonomous Systems*, 75, 661–670.

- Berge, O., Borge, R. E., Fjuk, A., Kaasbøll, J., & Samuelsen, T. (2003). Learning object-oriented programming. In *Norsk informatik konferanse*. Retrieved from <https://telearn.archives-ouvertes.fr/hal-00190184/> on 7 December 2016.
- Berman, R. A. (2014). Engaging students requires a renewed focus on teaching. *Chronicle of Higher Education*, 61(3), 28–30.
- Bers, M. U., Flannery, L., Kazakoff, E. R., & Sullivan, A. (2014). Computational thinking and tinkering: Exploration of an early childhood robotics curriculum. *Computers & Education*, 72, 145–157.
- Brennan, K. A. (2013). *Best of both worlds: Issues of structure and agency in computational creation, in and out of school*. Unpublished Doctoral dissertation, Massachusetts Institute of Technology, Cambridge.
- Bucci, P., Long, T. J., & Weide, B. W. (2001). Do we really teach abstraction?. Retrieved from <http://dl.acm.org/citation.cfm?id=364531> on 7 November 2016.
- Çakıroğlu, Ü., Sarı, E., & Akkan, Y. (2011). The view of the teachers about the contribution of teaching programming to the gifted students in the problem solving. In *5th International computer & instructional technologies symposium* (pp. 22–24).
- Chang, C. K. (2014). Effects of using Alice and Scratch in an introductory programming course for corrective instruction. *Journal of Educational Computing Research*, 51(2), 185–204.
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Hillsdale, NJ: Erlbaum.
- Cole, P. G., & Chan, L. (1994). *Teaching principles and practice*. Upper Saddle River: Prentice Hall.
- Denner, J., Werner, L., & Ortiz, E. (2012). Computer games created by middle school girls: Can they be used to measure understanding of computer science concepts? *Computers & Education*, 58(1), 240–249.
- Durak, H. (2016). *Design and development of an instructional program for teaching programming process to gifted students*. Unpublished Doctoral dissertation, Gazi University, Ankara.
- Esteves, M., & Mendes, A. J. (2004). A simulation tool to help learning of object oriented programming basics. In *34th Annual frontiers in education, 2004. FIE 2004* (pp. F4C–7). IEEE.
- Fesakis, G., & Serafeim, K. (2009). Influence of the familiarization with scratch on future teachers' opinions and attitudes about programming and ICT in education. *ACM SIGCSE Bulletin*, 41(3), 258–262.
- Fessakis, G., Gouli, E., & Mavroudi, E. (2013). Problem solving by 5–6 years old kindergarten children in a computer programming environment: A case study. *Computers & Education*, 63, 87–97.
- Fields, D. A., Giang, M. T., Kafai, Y. B. (2014). Programming in the wild: Patterns of computational participation in the Scratch online social networking forum. In *Proceedings of the 9th workshop in primary and secondary computing education* (pp. 2–11). New York, NY: ACM. Retrieved from <http://doi.acm.org/10.1145/2670757.2670768>.
- Fields, D. A., Searle, K. A., Kafai, Y. B., & Min, H. S. (2012). Debuggems to assess student learning in e-textiles. In *Proceedings of the 43rd ACM technical symposium on computer science education* (pp. 699–699). ACM.
- Forte, A. & Guzdial, M. (2014). Computers for communication, not calculation: media as a motivation and context for learning. In *Proceedings of the 37th annual Hawaii international conference on system sciences (HICSS'04)—track 4* (p. 40096.1).
- Fredricks, J. A., Blumenfeld, P. C., & Paris, A. H. (2004). School engagement: Potential of the concept, state of the evidence. *Review of educational research*, 74(1), 59–109.
- García-Peñalvo, F. J. (2016). A brief introduction to TACCLE 3—coding European project. In *International symposium on computers in education (SIIE), 2016* (pp. 1–4). IEEE.
- Gomes, A., & Mendes, A. J. (2007). Learning to program—difficulties and solutions. In *International conference on engineering education—ICEE*. Retrieved from https://www.researchgate.net/profile/Anabela_Gomes2/publication/228328491_Learning_to_program_-_difficulties_and_solutions/links/02e7e52389017b9984000000.pdf on 7 November 2016.
- Grover, S., & Pea, R. (2013). Computational thinking in K-12: A review of the state of the field. *Educational Researcher*, 42(1), 38–43.
- Hill, C. (2015). *Programming environments for children: Creating a language that grows with you*. Unpublished Doctoral dissertation, University of California, Santa Barbara, ABD.
- Hongwarittorn, N., & Krairit, D. (2010). Effects of program visualization (jeliot3) on students' performance and attitudes towards java programming. In *The spring 8th international conference on computing, communication and control technologies* (pp. 6–9).
- Katai, Z., & Toth, L. (2010). Technologically and artistically enhanced multi-sensory computer-programming education. *Teaching and teacher education*, 26(2), 244–251.
- Kayabaşı, E. (2016). *Prospective teachers' experiences on Alice: Programming in 3D environment*. Unpublished MA thesis, Uludağ University, Bursa.
- Kizilkaya, G., & Askar, P. (2009). The development of a reflective thinking skill scale towards problem solving. *Education and Science*, 34(154), 82.

- Korkmaz, Ö., Çakır, R., & Özden, M. Y. (2015). Computational thinking levels scale (CTLIS) adaptation for secondary school level. *Gazi Journal of Educational Science*, 1(2), 143–162.
- Korkmaz, Ö., Çakır, R., & Özden, M. Y. (2017). A validity and reliability study of the Computational Thinking Scales (CTS). *Computers in Human Behavior*, 72, 558–569.
- Kuh, G. D. (2009). What student affairs professionals need to know about student engagement. *Journal of college student development*, 50(6), 683–706.
- Lau, W. W., & Yuen, A. H. (2011). Modelling programming performance: Beyond the influence of learner characteristics. *Computers & Education*, 57(1), 1202–1213.
- Lavonen, J. M., Meisaloa, V. P., Lattua, M., & Sutinenb, E. (2003). Concretising the programming task: A case study in a secondary school. *Computers & Education*, 40(2), 115–135.
- Law, K. M., Lee, V. C., & Yu, Y. T. (2010). Learning motivation in e-learning facilitated computer programming courses. *Computers & Education*, 55(1), 218–228.
- Lawanto, K., Close, K., Ames, C., & Brasiel, S. (2017). Exploring strengths and weaknesses in middle school students' computational thinking in scratch. In P. Rich & C. Hodges (Eds.), *Emerging research, practice, and policy on computational thinking. Educational communications and technology: Issues and innovations*. Cham: Springer.
- Lippmann, S. (2013). Facilitating class sessions for ego-piercing engagement. *New Directions for Teaching and Learning*, 2013(135), 43–48.
- Lye, S. Y., & Koh, J. H. L. (2014). Review on teaching and learning of computational thinking through programming: What is next for K-12? *Computers in Human Behavior*, 41, 51–61.
- Maloney, J., Resnick, M., Rusk, N., Silverman, B., & Eastmond, E. (2010). The scratch programming language and environment. *ACM Transactions on Computing Education (TOCE)*, 10(4), 16.
- Margulieux, L. E., Guzdial, M., & Catrambone, R. (2012). Subgoal-labeled instructional material improves performance and transfer in learning to develop mobile applications. In *Proceedings of the ninth annual international conference on International computing education research* (pp. 71–78). ACM.
- Medlock-Walton, P., Harms, K. J., Kraemer, E. T., Brennan, K., & Wendel, D. (2014). Blocks-based programming languages: Simplifying programming for different audiences with different goals. In *Proceedings of the 45th ACM technical symposium on computer science education* (pp. 545–546). ACM.
- Mertler, C. A., & Vannatta, R. A. (2002). A guide to multivariate techniques. In C. A. Mertler (Ed.), *Advanced and multivariate statistical methods* (2nd ed.). Los Angeles, CA: Pyrczak Publishing.
- Moreno, J. (2012). Digital competition game to improve programming skills. *Journal of Educational Technology & Society*, 15(3), 288–297.
- Noble, J. (2013). *Building a LEGO-based robotics platform for a 3rd grade classroom*. Unpublished Doctoral dissertation, Tufts University.
- Özden, M. Y. (2015). Computational thinking. <http://myozden.blogspot.com.tr/2015/06/computational-thinking-bilgisayarca.html>. Accessed 15 June 2017.
- Papadakis, S., Kalogiannakis, M., Orfanakis, V., & Zaranis, N. (2014). Novice programming environments. Scratch & app inventor: A first comparison. In *Proceedings of the 2014 workshop on interaction design in educational environments* (p. 1). ACM.
- Perkins, D. N., & Salomon, G. (1992). Transfer of learning. *International encyclopedia of education*, 2, 6452–6457.
- Portelance, D. J., & Bers, M. U. (2015). Code and tell: Assessing young children's learning of computational thinking using peer video interviews with ScratchJr. In *Proceedings of the 14th international conference on interaction design and children* (pp. 271–274). ACM.
- Prensky, M., & Berry, B. D. (2001). Do they really think differently. *On the Horizon*, 9(6), 1–9.
- Psycharis, S., & Kallia, M. (2017). The effects of computer programming on high school students' reasoning skills and mathematical self-efficacy and problem solving. *Instructional Science*, 45(5), 583–602.
- Resnick, M., Maloney, J., Monroy-Hernández, A., Rusk, N., Eastmond, E., Brennan, K., et al. (2009). Scratch: Programming for all. *Communications of the ACM*, 52(11), 60–67.
- Robins, A., Rountree, J., & Rountree, N. (2003). Learning and teaching programming: A review and discussion. *Computer Science Education*, 13(2), 137–172.
- Rushkoff, D. (2010). *Program or be programmed: Ten commands for a digital age*. New York, NY: O/R Books.
- Sáez-López, J. M., Román-González, M., & Vázquez-Cano, E. (2016). Visual programming languages integrated across the curriculum in elementary school: A two year case study using “Scratch” in five schools. *Computers & Education*, 97, 129–141.
- Sarıtepeci, M. (2012). *Investigation of the effectiveness of digital storytelling method in social studies course*. Unpublished MA thesis, Gazi University, Ankara.

- Sarıtepeci, M., & Durak, H. (2017). Analyzing the effect of block and robotic coding activities on computational thinking in programming education. In I. Koleva & G. Duman (Eds.), *Educational research and practice* (pp. 490–501), Chapter 49 Sofia: St. Kliment Ohridski University Press.
- Shermis, S. S. (1992). *Critical thinking: Helping students learn reflectively.*, ERIC clearinghouse on reading and communication skills Bloomington, IN: Indiana University.
- Strampel, K., & Oliver, R. (2007). Using technology to foster reflection in higher education. In *ICT: Providing choices for learners and learning. Proceedings ascilite Singapore 2007*.
- Trees, F. P. (2010). *A meta-analysis of pedagogical tools used in introductory programming courses*. Doctoral dissertation, Pace University.
- Utting, I., Cooper, S., Kölling, M., Maloney, J., & Resnick, M. (2010). Alice, greenfoot, and scratch—A discussion. *ACM Transactions on Computing Education (TOCE)*, 10(4), 17.
- Wang, T. C., Mei, W. H., Lin, S. L., Chiu, S. K., & Lin, J. M. C. (2009). Teaching programming concepts to high school students with Alice. In *39th IEEE frontiers in education conference, 2009. FIE'09* (pp. 1–6). IEEE.
- Werner, L., Denner, J., & Campe, S. (2015). Children programming games: A strategy for measuring computational learning. *ACM Transactions on Computing Education (TOCE)*, 14(4), 1–22.
- Werner, L., Denner, J., Campe, S., & Kawamoto, D. C. (2012). The fairy performance assessment: measuring computational thinking in middle school. In *Proceedings of the 43rd ACM technical symposium on computer science education* (pp. 215–220). ACM.
- Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3), 33–35.
- Wing, J. (2011). Research notebook: Computational thinking—What and why? *The Link Magazine*, Spring. Carnegie Mellon University, Pittsburgh. <http://link.cs.cmu.edu/article.php?a=600>. Accessed 12 May 2017.
- Winslow, L. E. (1996). Programming pedagogy—A psychological overview. *ACM Sigcse Bulletin*, 28(3), 17–22.
- Yildiz Durak, H. (2018). Digital story design activities used for teaching programming effect on learning of programming concepts, programming self-efficacy, and participation and analysis of student experiences. *Journal of Computer Assisted learning*. <https://doi.org/10.1111/jcal.12281>.
- Yıldız Durak, H., & Güyer, T. (2018). Design and development of an instructional program for teaching programming processes to gifted students using scratch. In J. Cannaday (Ed.), *Curriculum development for gifted education programs* (pp. 61–99). Hershey: IGI Global.